

```

#include <assert.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "group.h"
#include "maps.h"
#include "linked_list.h"

```

```

int *zero_array(int n) {
    int *res = malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        res[i] = 0;
    }
    return res;
}

```

```

void group_name(group_t *g, char *s) {
    g->name = s;
}

```

```

void add_sum_to_decomp(int x, int y, decomp_t* d) {
    int* x_decomp = d->basis_decomps[x];
    int* y_decomp = d->basis_decomps[y];
    int z = (x+y) % d->n;
    int* z_decomp = d->basis_decomps[z];
    for (int i = 0; i < d->n; i++) {
        if (x_decomp[i] == -1 || y_decomp[i] == -1) {
            z_decomp[i] = -1;
        } else {
            z_decomp[i] = (x_decomp[i] + y_decomp[i]) % d->n;
        }
    }
}

```

```

group_t *group_empty(int n) {
    group_t *new_empty_group = malloc(sizeof(group_t));
    new_empty_group->n = n;
    new_empty_group->table = malloc(n * sizeof(int *));
    for (int i = 0; i < n; i++) {
        new_empty_group->table[i] = zero_array(n);
    }
    return new_empty_group;
}

```

```

void group_free(group_t *g) {
    assert(g != NULL && "[ ] Pointeur nul.\n");
    int n = g->n;
    for (int i = 0; i < n; i++) {
        free(g->table[i]);
    }
    free(g->table);
    free(g);
}

```

```

gdata_t* create_empty_gdata() {
    gdata_t* gdata = malloc(sizeof(gdata_t));
    return gdata;
}

```

```

// WE GIVE THE POINTERS WE DON'T COPY.
gdata_t* parse_entries(list_t* gen, gentabl_t* rel) {
    gdata_t* gdata = create_empty_gdata();
    gdata->gen = gen;
    gdata->rel = rel;
    return gdata;
}

```

```

}

void free_gdata(gdata_t* gdata) {
    assert(gdata != NULL);
    free_list(gdata->gen);
    free_gentabl(gdata->rel);
    free(gdata);
}

list_t* get_generator(gdata_t* gdata) {
    assert(gdata != NULL);
    return gdata->gen;
}

gentabl_t* get_rel(gdata_t* gdata) {
    assert(gdata != NULL);
    return gdata->rel;
}

void group_print(group_t *g) {
    assert(g != NULL && "[_] Pointeur nul.\n");
    int n = g->n;
    if (g->name == NULL || g->name[0] == '\0') {
        printf("Groupe d'ordre: %d\n", n);
    } else {
        printf("Groupe %s d'ordre %d\n", g->name, n);
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", g->table[i][j]);
        }
        printf("\n");
    }
}

group_t *from_matrix(int **m) {
    assert(m != NULL && "[_] Pointeur nul.\n");
    int n = sizeof(*m[0]) / sizeof(int);
    group_t *res = group_empty(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            res->table[i][j] = m[i][j];
        }
    }
    return res;
}

group_t *copy_group(group_t *g) {
    assert(g != NULL && "[_] Pointeur nul.\n");
    int n = g->n;
    group_t *copy = group_empty(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            copy->table[i][j] = g->table[i][j];
        }
    }
    return copy;
}

int calculate(group_t* g, int x, int y) {
    if (x == -1) {
        return y;
    }
    return g->table[x][y];
}

bool is_neutral(group_t* g, int x) {

```

```

assert(g != NULL);
assert(x >= 0 && x < g->n);
int res;
if (x < g->n - 1) {
    res = calculate(g, x, x+1);
    return (res == x+1);
} else {
    res = calculate(g, x, 0);
    return (res == 0);
}
}

int get_neutral(group_t* g) {
    assert(g != NULL);
    for (int i = 0; i < g->n; i++) {
        if (is_neutral(g, i)) {
            return i;
        }
    }
    printf("Error\n.");
    return -1;
}

int calculate_order(group_t* g, int x) {
    assert(g != NULL);
    assert(x >= 0 && x < g->n);
    if (is_neutral(g, x)) {
        return 0;
    }
    int curr = x;
    int order = 1;
    while (!is_neutral(g, curr)) {
        curr = calculate(g, curr, x);
        order++;
    }

    return order;
}

int* get_orders_list(group_t* g) {
    assert(g != NULL);
    int* orders = malloc(g->n*sizeof(int));
    for (int x = 0; x < g->n; x++) {
        orders[x] = calculate_order(g, x);
    }
    return orders;
}

list_t* get_cyclic_subgroup_element(group_t* g, int x, int e) {
    assert(g != NULL);
    assert(0 <= x < g->n);
    assert(is_neutral(g, e));
    int curr = x;
    list_t* cyclic = create_empty_list();
    push(cyclic, e);
    while (!is_neutral(g, curr)) {
        push(cyclic, curr);
        curr = calculate(g, curr, x);
    }

    return cyclic;
}

int calc_generated_subgroup(group_t* g, int new_gen, list_t* generators, gentabl_t*
decomp_tbl) {
    assert(decomp_tbl->size == g->n);
    assert(generators != NULL && g != NULL);

```

```

    if (new_gen >= g->n) {
        return -1;
    }

    push(generators, new_gen);

    list_t* single = create_single(new_gen);
    copy_at_index(decomp_tbl, new_gen, single);
    free_list(single);

    // Variable pour le candidat d'un autre générateur
    int next_gen = new_gen+1;

    int to_add;
    list_t* to_add_decomp;
    for (int elt = 0; elt < g->n; elt++) {
        if (!is_empty_index(decomp_tbl, elt)) {
            to_add = calculate(g, elt, new_gen);
            if (is_empty_index(decomp_tbl, to_add)) {
                // Calculer la décomposition de to_add.
                to_add_decomp = get_copy_index_val(decomp_tbl, elt);
                append(to_add_decomp, new_gen);
                copy_at_index(decomp_tbl, to_add, to_add_decomp);

                free_list(to_add_decomp);
            }
            if (next_gen < g->n && !is_empty_index(decomp_tbl, next_gen)) {
                next_gen++;
            }
        }
    }

    return next_gen;
}

gdata_t* calc_gen_and_rel(group_t* g) {
    list_t* generators = create_empty_list();
    gentabl_t* relations = create_empty_gentabl(g->n);

    int curr_gen = 0;
    while (curr_gen < g->n) {
        curr_gen = calc_generated_subgroup(g, curr_gen, generators, relations);
    }

    gdata_t* result = parse_entries(generators, relations);
    return result;
}

void print_array(int* arr, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

bool are_isomorphic(group_t* g1, group_t* g2) {
    assert(g1 != NULL && g2 != NULL);
    assert(g1->n == g2->n);

    // Précalculs sur les données de g1.
    gdata_t* g1_gdata = calc_gen_and_rel(g1);
    gentabl_t* rel = g1_gdata->rel;
    list_t* gen = g1_gdata->gen;

    int n = g1->n;
    int m = gen->length;

```

```

int count_perm = 0;
int* perm;
// Nombres de permutations.
llong num_perm = factorial(m);

// Table des coefficients binomiaux: binom[k][p] = k parmi p
int** binom = get_binomial(n);

int count = 0;
int curr_subset_num = 0;
int num_subsets = binom[m][n];
int* induced_map;
bool found = false;

while (curr_subset_num < num_subsets && !found) {
    int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
    print_array(chosen_images, m);
    int* curr_permutated_images;
    int curr_perm_num = 0;
    while (curr_perm_num < num_perm && !found) {
        perm = get_kth_permutation_rec(m, curr_perm_num);
        curr_permutated_images = get_from_permutation(perm, chosen_images, m);
        induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
        if (is_isomorphism(g1, g1_gdata, g2, induced_map, n)) {
            found = true;
            printf("[+] Isomorphisme trouvé.\n");
            print_map(induced_map, n);

            free(induced_map);
            free(perm);
            free(curr_permutated_images);
            break;
        }

        free(induced_map);
        free(perm);
        free(curr_permutated_images);
        curr_perm_num++;
        count++;
    }

    free(chosen_images);
    curr_subset_num++;
}

free_gdata(g1_gdata);
free_matrix(binom, n);

return found;
}

bool are_isomorphic_with_order(group_t* g1, group_t* g2) {
    assert(g1 != NULL && g2 != NULL);
    assert(g1->n == g2->n);

    gdata_t* g1_gdata = calc_gen_and_rel(g1);
    gentabl_t* rel = g1_gdata->rel;
    int* orders_g1 = get_orders_list(g1);
    int* orders_g2 = get_orders_list(g2);

    list_t* gen = g1_gdata->gen;

    int n = g1->n;
    int m = gen->length;

```

```

int count_perm = 0;
int* perm;
llong num_perm = factorial(m);

int** binom = get_binomial(n);

int count = 0;

int curr_subset_num = 0;
int num_subsets = binom[m][n];
int* induced_map;
bool found = false;

while (curr_subset_num < num_subsets && !found) {
    int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
    int* curr_permutated_images;
    int curr_perm_num = 0;
    while (curr_perm_num < num_perm && !found) {
        perm = get_kth_permutation_rec(m, curr_perm_num);
        curr_permutated_images = get_from_permutation(perm, chosen_images, m);
        induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
        if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2, induced_map, n)) {
            found = true;

            free(induced_map);
            free(perm);
            free(curr_permutated_images);
            break;
        }

        free(induced_map);
        free(perm);
        free(curr_permutated_images);
        curr_perm_num++;
        count++;
    }

    free(chosen_images);
    curr_subset_num++;
}

free(orders_g1);
free(orders_g2);
free_gdata(g1_gdata);
free_matrix(binom, n);

return found;
}

bool are_isomorphic_with_order_custom_gen(group_t* g1, group_t* g2, int* generators, int
num_gen) {
    assert(g1 != NULL && g2 != NULL);
    assert(g1->n == g2->n);
    assert(generators != NULL && num_gen > 0);

    list_t* gen = create_empty_list();
    gentabl_t* rel = create_empty_gentabl(g1->n);

    for (int i = 0; i < num_gen; i++) {
        assert(generators[i] >= 0 && generators[i] < g1->n);
        calc_generated_subgroup(g1, generators[i], gen, rel);
    }

    gdata_t* g1_gdata = create_empty_gdata();
    g1_gdata->gen = gen;
    g1_gdata->rel = rel;

```

```

int* orders_g1 = get_orders_list(g1);
int* orders_g2 = get_orders_list(g2);

int n = g1->n;
int m = gen->length;

int* perm;
llong num_perm = factorial(m);

int** binom = get_binomial(n);

int count = 0;
int curr_subset_num = 0;
int num_subsets = binom[m][n];
int* induced_map;
bool found = false;

while (curr_subset_num < num_subsets && !found) {
    int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
    int* curr_permutated_images;
    int curr_perm_num = 0;

    while (curr_perm_num < num_perm && !found) {
        perm = get_kth_permutation_rec(m, curr_perm_num);
        curr_permutated_images = get_from_permutation(perm, chosen_images, m);
        induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);

        if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2, induced_map, n)) {
            found = true;
            printf("[+] Isomorphisme trouvé.\n");
            print_map(induced_map, n);

            free(induced_map);
            free(perm);
            free(curr_permutated_images);
            break;
        }

        free(induced_map);
        free(perm);
        free(curr_permutated_images);
        curr_perm_num++;
        count++;
    }

    free(chosen_images);
    curr_subset_num++;
}

free(orders_g1);
free(orders_g2);
free_gdata(g1_gdata);
free_matrix(binom, n);

return found;
}

bool are_isomorphic_random(group_t* g1, group_t* g2) {
    assert(g1 != NULL && g2 != NULL);
    assert(g1->n == g2->n);

    gdata_t* g1_gdata = calc_gen_and_rel(g1);
    gentabl_t* rel = g1_gdata->rel;
    int* orders_g1 = get_orders_list(g1);
    int* orders_g2 = get_orders_list(g2);

    list_t* gen = g1_gdata->gen;

```

```

int n = g1->n;
int m = gen->length;

int count_perm = 0;
int* perm;
llong num_perm = factorial(m);

int** binom = get_binomial(n);

int count = 0;

int curr_subset_num = 0;
int num_subsets = binom[m][n];
int* induced_map;
bool found = false;

while (curr_subset_num < num_subsets && !found) {
    int* chosen_images = get_array_from_subset_binom(curr_subset_num, m, n, binom);
    int* curr_permutated_images;
    int curr_perm_num = 0;
    while (curr_perm_num < num_perm && !found) {
        perm = get_kth_permutation_rec(m, curr_perm_num);
        curr_permutated_images = get_from_permutation(perm, chosen_images, m);
        induced_map = get_full_map(g1_gdata, g2, curr_permutated_images, m);
        if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2, induced_map, n)) {
            found = true;
            printf("[+] Isomorphisme trouvé.\n");
            print_map(induced_map, n);

            free(induced_map);
            free(perm);
            free(curr_permutated_images);
            break;
        }

        free(induced_map);
        free(perm);
        free(curr_permutated_images);
        curr_perm_num++;
        count++;
    }

    free(chosen_images);
    curr_subset_num++;
}

free(orders_g1);
free(orders_g2);
free_gdata(g1_gdata);
free_matrix(binom, n);

return found;
}

bool are_isomorphic_probabilistic(group_t* g1, group_t* g2, int max_attempts) {
    assert(g1 != NULL && g2 != NULL);
    assert(g1->n == g2->n);

    srand(time(NULL));

    gdata_t* g1_gdata = calc_gen_and_rel(g1);
    gentabl_t* rel = g1_gdata->rel;
    list_t* gen = g1_gdata->gen;

    int n = g1->n;
    int m = gen->length;

```



```

int* orders_g1 = get_orders_list(g1);
int* orders_g2 = get_orders_list(g2);

int* gen_orders = malloc(m * sizeof(int));
int gen_idx = 0;
for (node_t* c = gen->head; c != NULL; c = c->next) {
    gen_orders[gen_idx++] = orders_g1[c->val];
}

bool found = false;
int* induced_map = NULL;

for (int attempt = 0; attempt < max_attempts && !found; attempt++) {
    int* chosen_images = get_random_subset(m, n);

    bool order_compatible = true;
    for (int i = 0; i < m; i++) {
        if (gen_orders[i] != orders_g2[chosen_images[i]]) {
            order_compatible = false;
            break;
        }
    }

    if (!order_compatible) {
        free(chosen_images);
        continue;
    }

    int* perm = get_random_permutation(m);
    int* permuted_images = get_from_permutation(perm, chosen_images, m);

    induced_map = get_full_map(g1_gdata, g2, permuted_images, m);

    if (is_isomorphism_with_order(g1, g1_gdata, g2, orders_g1, orders_g2, induced_map, n)) {
        found = true;
        printf("[+] Isomorphisme trouvé après %d itérations!\n", attempt + 1);
        print_map(induced_map, n);
    }

    free(chosen_images);
    free(perm);
    free(permuted_images);
    if (!found) {
        free(induced_map);
    }

    if ((attempt + 1) % 10000 == 0) {
        printf("Itération %d...\n", attempt + 1);
    }
}

free(gen_orders);
free(orders_g1);
free(orders_g2);
free_gdata(g1_gdata);

if (found) {
    free(induced_map);
}

return found;
}

```

```

bool are_isomorphic_hybrid(group_t* g1, group_t* g2, long long deterministic_threshold, int
probabilistic_attempts) {
    assert(g1 != NULL && g2 != NULL);

```

```

assert(g1->n == g2->n);

gdata_t* g1_gdata = calc_gen_and_rel(g1);
int n = g1->n;
int m = g1_gdata->gen->length;
free_gdata(g1_gdata);

int** binom = get_binomial(n);
long long num_subsets = binom[m][n];
free_matrix(binom, n);

llong num_perm = factorial(m);
long long total_candidates = num_subsets * num_perm;

if (total_candidates <= deterministic_threshold) {
    return are_isomorphic_with_order(g1, g2);
} else {
    return are_isomorphic_probabilistic(g1, g2, probabilistic_attempts);
}
}

bool are_isomorphic_adaptive(group_t* g1, group_t* g2) {
    assert(g1 != NULL && g2 != NULL);
    assert(g1->n == g2->n);

    int n = g1->n;
    gdata_t* g1_gdata = calc_gen_and_rel(g1);
    int m = g1_gdata->gen->length;
    free_gdata(g1_gdata);

    int** binom = get_binomial(n);
    long long total_space = binom[m][n] * factorial(m);
    free_matrix(binom, n);

    int attempts = (int)sqrt((double)total_space);
    if (attempts < 1000) attempts = 1000;
    if (attempts > 1000000) attempts = 1000000;

    return are_isomorphic_probabilistic(g1, g2, attempts);
}

```